

UFUG 2106 : Discrete Mathematics

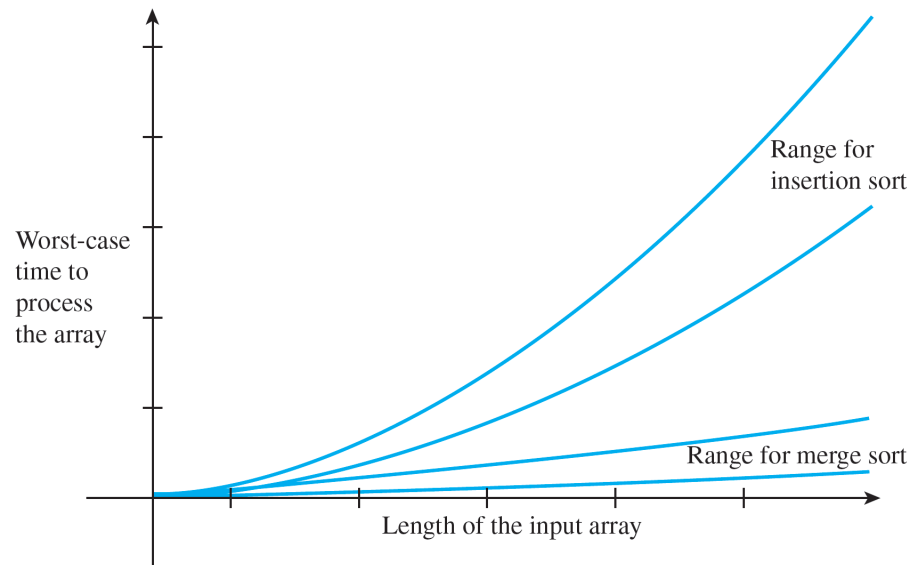
Chapter 11: Analysis of Algorithm Efficiency

Outline

1. Big-O, Big-Omega, and Big-Theta Notations
2. Application: Analysis of Algorithm Efficiency: I
3. Application: Analysis of Algorithm Efficiency : II

Big-O, Big-Omega, and Big-Theta Notations

The symbols Θ and Ω are the uppercase Greek letters theta and omega. The Θ -notation introduced in this section is determined by ranges such as those shown in Figure 11.2.1.



•Figure 11.2.1

Big-O, Big-Omega, and Big-Theta Notations

It is related to two other notations: Ω -notation and Θ -notation. The oldest of the notations, O -notation (read “big- O notation”), was introduced by the German mathematician Paul Bachmann in 1894 in a book on analytic number theory, and it was the first to be used to compare efficiencies of algorithms.

As you will see, however, using O -notation alone can produce ambiguous or even misleading results.

Big-O, Big-Omega, and Big-Theta Notations

As a response, in 1976 Donald Knuth, a pioneer in the analysis of algorithms, introduced the Ω - and Θ -notations so that the growth of functions could be compared with greater precision.

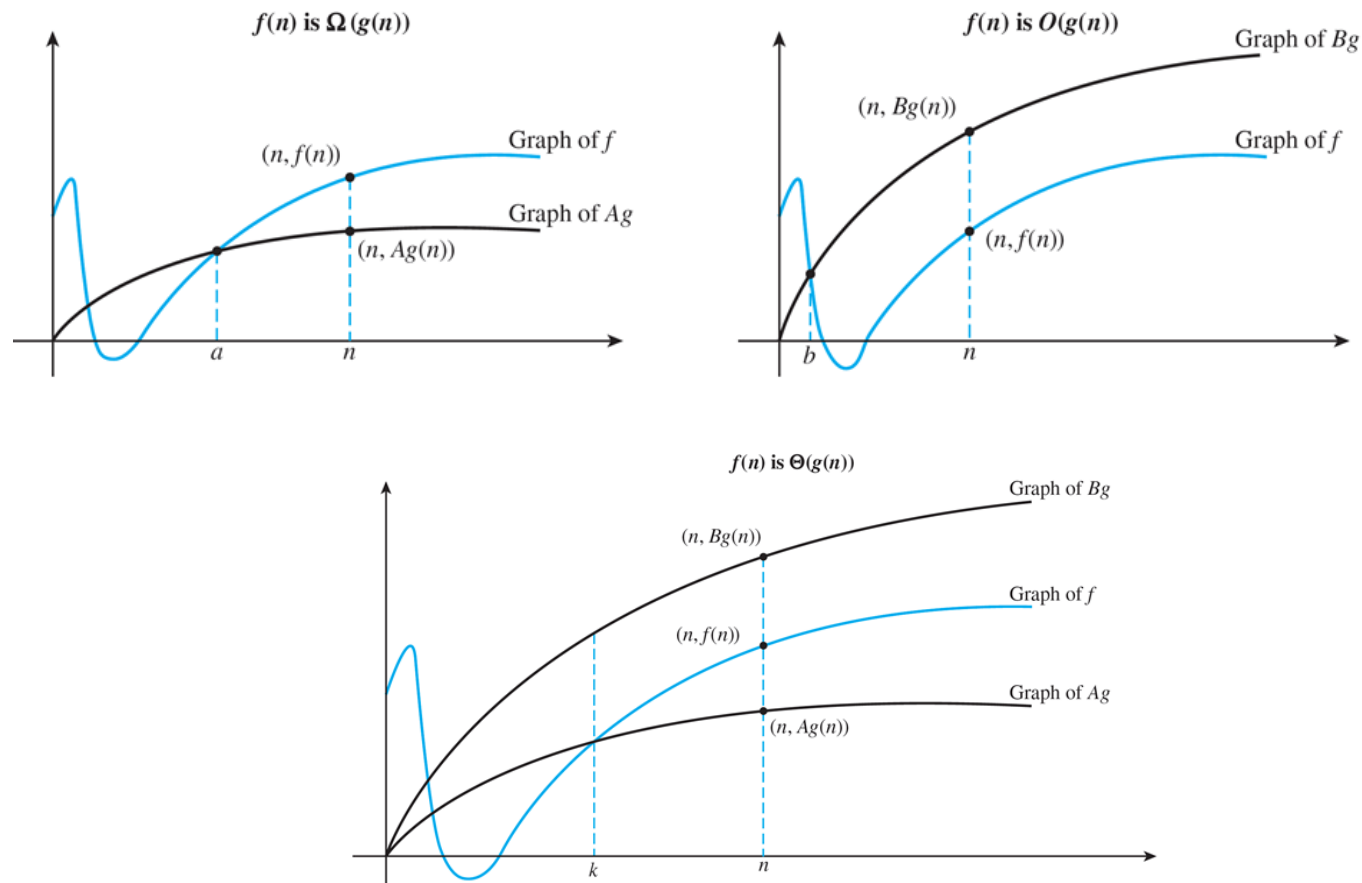
The idea of the notations is this: Suppose f and g are real-valued functions of an integer variable n and suppose $g(n) \geq 0$ for every integer n greater than some positive real number.

Big-O, Big-Omega, and Big-Theta Notations

1. If, for sufficiently large values of n , the values of f are greater than those of a positive multiple of g , then f is of order at least g , written “ $f(n)$ is $\Omega(g(n))$.”
2. If, for sufficiently large values of n , the values of f are positive and less than those of a positive multiple of g , then f is of order at most g , written “ $f(n)$ is $O(g(n))$.”
3. If, for sufficiently large values of n , the values of f are bounded both above and below by those of positive multiples of g , then f is of order g , written “ $f(n)$ is $\Theta(g(n))$.”

Big-O, Big-Omega, and Big-Theta Notations

These relationships are illustrated in Figure 11.2.2.



•Figure 11.2.2

Big-O, Big-Omega, and Big-Theta Notations

Definition

Let f and g be real-valued functions defined on the same set of nonnegative integers, with $g(n) \geq 0$ for every integer $n \geq r$, where r is a positive real number. Then

1. f is of order at least g , written $f(n)$ is $\Omega(g(n))$ (f of n is big-Omega of g of n), if, and only if, there exist positive real numbers A and $a \geq r$ such that

$$Ag(n) \leq f(n) \quad \text{for every integer } n \geq a.$$

2. f is of order at most g , written $f(n)$ is $O(g(n))$ (f of n is big- O of g of n), if, and only if, there exist positive real numbers B and $b \geq r$ such that

$$0 \leq f(n) \leq Bg(n) \quad \text{for every integer } n \geq b.$$

3. f is of order g , written $f(n)$ is $\Theta(g(n))$ (f of n is big-Theta of g of n), if, and only if, there exist positive real numbers A , B , and $k \geq r$ such that

$$Ag(n) \leq f(n) \leq Bg(n) \quad \text{for every integer } n \geq k.$$

Example 11.2.1 – Translating to Θ -Notation

Use Θ -notation to express the statement

$$4n^6 \leq 17n^6 - 45n^3 + 2n + 8 \leq 30n^6 \text{ for every integer } n \geq 3.$$

Example 11.2.2 – Translating to Ω - and O -Notations and Deducing Θ -Notation

a. Use Ω -notation to express the statement

$$\frac{11}{4}n^2 \leq 3\left(\left\lfloor \frac{n}{4} \right\rfloor\right)^2 + 5n^2 \quad \text{for every integer } n \geq 2.$$

b. Use O -notation to express the statement

$$0 \leq 3\left(\left\lfloor \frac{n}{4} \right\rfloor\right)^2 + 5n^2 \leq 6n^2 \quad \text{for every integer } n \geq 1.$$

c. Justify the statement:

$$3\left(\left\lfloor \frac{n}{4} \right\rfloor\right)^2 + 5n^2 \text{ is } \Theta(n^2).$$

Big-O, Big-Omega, and Big-Theta Notations

Theorem 11.2.1 Relation among O -, Ω -, and Θ -Notations

If f and g are real-valued functions defined on the same set of nonnegative integers, and if $f(n) \geq 0$ and $g(n) \geq 0$ for every integer $n \geq r$, where r is a positive real number, then $f(n)$ is $\Theta(g(n))$ if, and only if, $f(n)$ is $\Omega(g(n))$ and $f(n)$ is $O(g(n))$.

Orders of Power Functions

The functions that are most commonly used for comparing algorithm efficiencies are power functions, such as $n^{1/2}$, n , n^2 , and n^3 , and combinations involving a power function and an exponential or logarithmic function, such as 2^n , $\log(n)$, $n \log(n)$, and $n^2 \log(n)$.

These functions arise naturally in the analysis of algorithms, but, for large values of n , their sizes are dramatically different. In this section we focus on power functions.

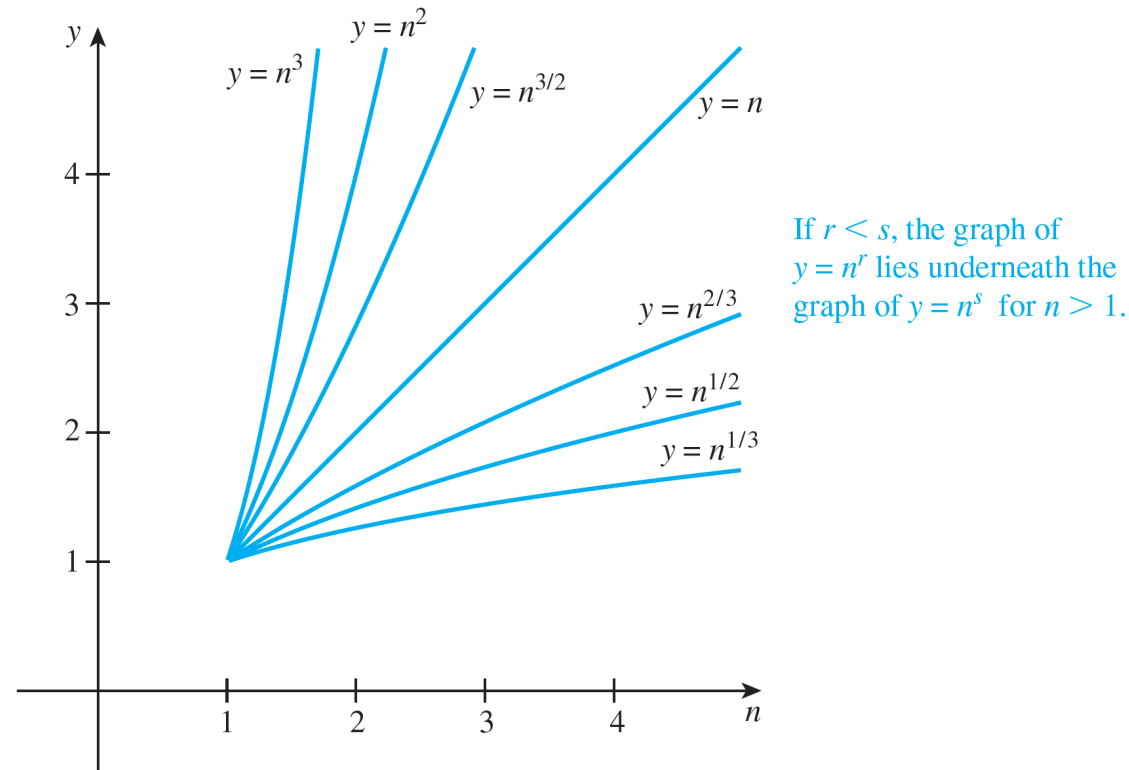
Orders of Power Functions

Theorem 11.2.2 For any positive rational numbers r and s and any integer $n \geq 1$,

$$\text{if } r \leq s, \quad \text{then } n^r \leq n^s.$$

Orders of Polynomial Functions

The following two examples show how to use Theorem 11.2.2 to find a big- Ω and a big- O for some polynomial functions.



•Graphs of Powers of n for $n \geq 1$

•Figure 11.2.3

Example 11.2.3 – Finding a Big-Omega and a Big-O for a Polynomial Function with Nonnegative Coefficients

- a. Show that $15n^3 + 11n^2 + 9$ is $\Omega(n^3)$.
- b. Without using part (a), show that $15n^3 + 11n^2 + 9$ is $O(n^3)$.

Orders of Polynomial Functions

Let m be a nonnegative integer, let $P(n)$ be a polynomial of degree m , and suppose the coefficient a_m of n^m is positive.

To find big-Omega for $P(n)$: Let $A = \frac{1}{2}a_m$, and let a be the number obtained as follows:

1. Find the sum of the absolute values of all the coefficients of $P(n)$ except for a_m .
2. Multiply the result of step 1 by $\frac{2}{a_m}$.
3. Let a be the larger of the number 1 and the result of step 2.

Show that $An^m \leq P(n)$ for every integer $n \geq a$.

Example 11.2.5 – Finding a Big-Theta for a Polynomial Function

a. Show that $15n^3 + 11n^2 + 9$ is $\Theta(n^3)$.

b. Show that $n^4 - 5n - 8$ is $\Theta(n^4)$.

Orders of Polynomial Functions

Theorem 11.2.3 A Limit on What Can Be Inferred from Big-O

For any function f and positive real numbers r and s with $r < s$,

if $f(n)$ is $O(n^r)$ then $f(n)$ is $O(n^s)$.

Example 11.2.6 – A Caution about O-notation

Suppose a person finds that $f(n)$ is $O(n^5)$ and that $g(n)$ is $O(n^4)$. Because $4 < 5$, this person might conclude that the graph of g lies below the graph of f for large values of n .

Show that this is not necessarily the case by showing that there exist functions f and g such that $f(n)$ is $O(n^5)$ and $g(n)$ is $O(n^4)$, yet for large values of n the graph of f lies below, not above, the graph of g .

Example 11.2.7 – Showing That One Function Is Not Big-O of Some Other Function

Let $g(n) = n^4 - 5n - 8$. By Example 11.2.4, $g(n)$ is $\Omega(n^4)$. Show that $g(n)$ is not $O(n^r)$ for any positive real number $r < 4$.

Orders of Polynomial Functions

Theorem 11.2.4 Showing That a Big-O Relationship Does Not Hold

If f is a real-valued function defined on a set of nonnegative integers and $f(n)$ is $\Omega(n^m)$, where m is a positive integer, then $f(n)$ is not $O(n^p)$ for any positive real number $p < m$.

Theorem 11.2.5 On Polynomial Orders

If m is any integer with $m \geq 0$ and $a_0, a_1, a_2, \dots, a_m$ are real numbers with $a_m > 0$, then $a_m n^m + a_{m-1} n^{m-1} + \dots + a_1 n + a_0$ is $\Theta(n^m)$.

Example 11.2.8 – Calculating Polynomial Orders Using the Theorem on Polynomial Orders

Use the theorem on polynomial orders to find orders for the functions given by the following formulas.

a. $f(n) = 7n^5 + 5n^3 - n + 4$ for each positive integer n .

b. $g(n) = \frac{(n-1)(n+1)}{4}$ for each positive integer n .

Orders of Polynomial Functions

Theorem 11.2.6 Reciprocal Relationship between Ω - and O -notations

Let f and g be real-valued functions defined on the same set of nonnegative integers, and suppose there is a positive real number r such that $f(n) \geq 0$ and $g(n) \geq 0$ for every integer $n \geq r$. Then:

- a. If $f(n)$ is $\Omega(g(n))$, then $g(n)$ is $O(f(n))$.
- b. If $g(n)$ is $O(f(n))$, then $f(n)$ is $\Omega(g(n))$.

Theorem 11.2.7 Reflexive, Symmetric, and Transitive Properties of Θ -notation

Let f , g , and h be real-valued functions defined on the same set of nonnegative integers, and suppose there is a positive real number r such that $f(n) \geq 0$, $g(n) \geq 0$ and $h(n) \geq 0$, for every integer $n \geq r$. Then:

- a. $f(n)$ is $\Theta(f(n))$.
- b. If $f(n)$ is $\Theta(g(n))$, then $g(n)$ is $\Theta(f(n))$.
- c. If $f(n)$ is $\Theta(g(n))$ and $g(n)$ is $\Theta(h(n))$, then $f(n)$ is $\Theta(h(n))$.

Orders of Polynomial Functions

Theorem 11.2.8 Effect of Constants on Order Notations

Let f and g be real-valued functions defined on the same set of nonnegative integers, and suppose there is a positive real number r such that $f(n) \geq 0$ and $g(n) \geq 0$ for every integer $n \geq r$.

Then for every positive real number c :

- a. If $f(n)$ is $\Omega(g(n))$, then $cf(n)$ is $\Omega(g(n))$;
- b. If $f(n)$ is $O(g(n))$, then $cf(n)$ is $O(g(n))$;
- c. If $f(n)$ is $\Theta(g(n))$, then $cf(n)$ is $\Theta(g(n))$.

Theorem 11.2.9 Orders of Sums and Products of Functions

Let f_1, f_2, g_1 , and g_2 be real-valued functions defined on the same set of nonnegative integers, and suppose there is a positive real number r such that $f_1(n) \geq 0, f_2(n) \geq 0, g_1(n) \geq 0$, and $g_2(n) \geq 0$ for every integer $n \geq r$. Then:

- a. If $f_1(n)$ is $\Theta(g_1(n))$ and $f_2(n)$ is $\Theta(g_2(n))$, then $(f_1(n) + f_2(n))$ is $\Theta(g_1(n) + g_2(n))$.
- b. If $f_1(n)$ is $\Theta(g_1(n))$ and $f_2(n)$ is $\Theta(g_2(n))$, then $(f_1(n)f_2(n))$ is $\Theta(g_1(n)g_2(n))$.
- c. If $f_1(n)$ is $\Theta(g_1(n))$ and $f_2(n)$ is $\Theta(g_2(n))$ and if there is a real number s so that $g_1(n) \leq g_2(n)$ for every integer $n \geq s$, then $(f_1(n) + f_2(n))$ is $\Theta(g_2(n))$.

Outline

1. Big-O, Big-Omega, and Big-Theta Notations
2. Application: Analysis of Algorithm Efficiency: I
3. Application: Analysis of Algorithm Efficiency : II

Application Analysis of Algorithm Efficiency: I

Starting in the late 1940s a number of mathematicians and computer scientists contributed to developing formal techniques for analyzing computer algorithms.

Alan Turing may have been the first to suggest a concrete way for doing this.

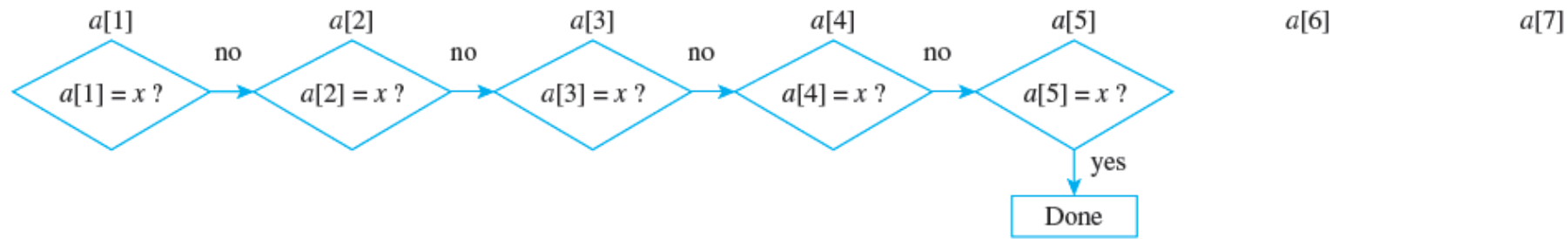
The Sequential Search Algorithm

The object of a search algorithm is to hunt through an array of data in an attempt to find a particular item x . In a sequential search, x is compared to the first item in the array, then to the second, then to the third, and so on.

The search is stopped if a match is found at any stage. On the other hand, if the entire array is processed without finding a match, then x is not in the array.

The Sequential Search Algorithm

An example of a sequential search is shown diagrammatically in Figure 11.3.1.



Sequential Search of $a[1], a[2], \dots, a[7]$ for x where $x = a[5]$

Figure 11.3.1

Example 11.3.1 – Best-, Worst, and Average-Case Orders for Sequential Search

Find best, worst-, and average-case orders for the sequential search algorithm from among the set of power functions.

Measuring the Efficiency of an Algorithm

Two aspects of algorithm efficiency are important: the amount of time required to execute the algorithm and the amount of memory space needed when it is run. In this chapter we introduce basic techniques for calculating time efficiency.

Similar techniques exist for calculating space efficiency. Occasionally, one algorithm may make more efficient use of time but less efficient use of memory space than another, forcing a trade-off based on the resources available to the user.

Measuring the Efficiency of an Algorithm

How can the time efficiency of an algorithm be calculated? The answer depends on several factors. One is the size of the set of data that is input to the algorithm; for example, it takes longer for a sort algorithm to process 1,000,000 items than 100 items. Consequently, the execution time of an algorithm is generally expressed as a function of its input size.

The analysis of an algorithm for time efficiency begins by trying to count the number of elementary operations that must be performed when the algorithm is executed with an input of size n (in the best case, worst case, or average case).

Measuring the Efficiency of an Algorithm

For instance, to compare two algorithms for evaluating a polynomial, the crucial issue is the number of additions and multiplications that are needed, whereas to compare two algorithms for searching a list to find a particular element, the important distinction is the number of comparisons that are required.

As is common, we will classify the following as elementary operations: addition, subtraction, multiplication, division, and comparisons that are indicated explicitly in an if-then statement using one of the relational symbols $<$, $\leq$, $>$, $\geq$, $=$, or $\neq$.

Measuring the Efficiency of an Algorithm

When algorithms are implemented in a particular programming language and run on a particular computer, some operations are executed faster than others, and, of course, there are differences in execution times from one machine to another.

In certain practical situations these factors are taken into account when deciding which algorithm or which machine to use to solve a particular problem.

Measuring the Efficiency of an Algorithm

In other cases, however, the machine is fixed, and rough estimates are all that are needed to determine the clear superiority of one algorithm over another.

Since each elementary operation is executed in time no longer than the slowest, the time efficiency of an algorithm is approximately proportional to the number of elementary operations required to execute the algorithm.

Measuring the Efficiency of an Algorithm

For example, to accomplish a job with an input size of 1,000,000, algorithm B requires almost 2 trillion more elementary operations than algorithm A.

One way to look at this is that algorithm B has to perform 100,000 operations for each single operation performed by algorithm A.

Measuring the Efficiency of an Algorithm

Definition

Let A be an algorithm.

1. Suppose the number of elementary operations performed when A is executed for an input of size n depends on n alone and not on the nature of the input data; say it equals $f(n)$. If $f(n)$ is $\Theta(g(n))$, we say that A is $\Theta(g(n))$ or A is of order $g(n)$.
2. Suppose the number of elementary operations performed when A is executed for an input of size n depends on the nature of the input data as well as on n .
 - a. Let $b(n)$ be the *minimum* number of elementary operations required to execute A for all possible input sets of size n . If $b(n)$ is $\Theta(g(n))$, we say that **in the best case, A is $\Theta(g(n))$ or A has a best-case order of $g(n)$.**
 - b. Let $w(n)$ be the *maximum* number of elementary operations required to execute A for all possible input sets of size n . If $w(n)$ is $\Theta(g(n))$, we say that **in the worst case, A is $\Theta(g(n))$ or A has a worst-case order of $g(n)$.**

Example 11.3.2 – Computing an Order of an Algorithm Segment

Assume n is a positive integer and consider the following algorithm segment:

```
 $p := 0, x := 2$   
for  $i := 2$  to  $n$   
     $p := (p + i) \cdot x$   
next  $i$ 
```

- Compute the actual number of elementary operations that are performed when this algorithm segment is executed.
- Use the theorem on polynomial orders to find an order for this algorithm segment.

Example 11.3.4 – When the Number of Iterations Depends on the Floor Function

Assume n is a positive integer and consider the following algorithm segment:

```
for  $i := \lfloor n/2 \rfloor$  to  $n$   
     $a := n - i$   
next  $i$ 
```

- Compute the actual number of elementary operations that are performed when this algorithm segment is executed.
- Use the theorem on polynomial orders to find an order for this algorithm segment.

The Insertion Sort Algorithm

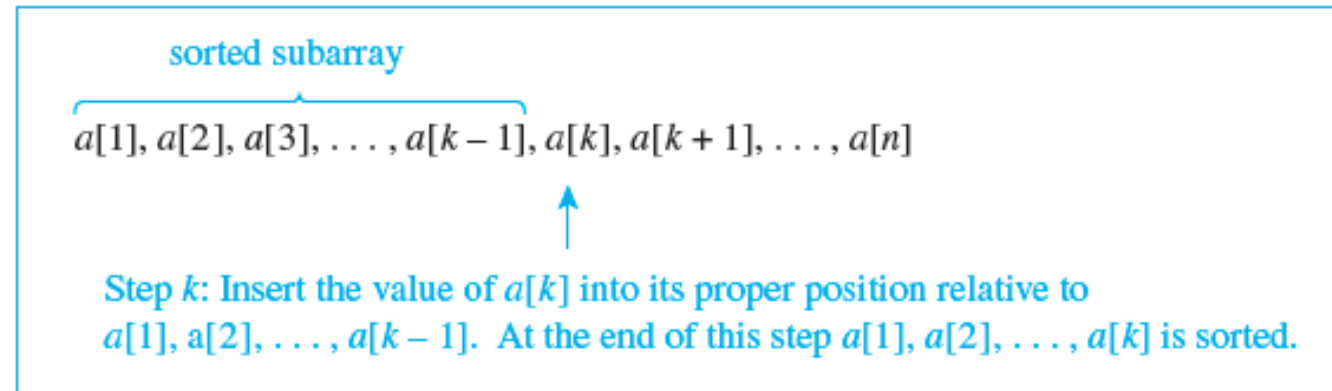
Insertion sort is an algorithm for arranging the items in an array into ascending order. Initially, the second item is compared to the first. If the second item is less than the first, their values are interchanged, and as a result the first two array items are in ascending order.

The idea of the algorithm is gradually to lengthen the section of the array that is known to be in ascending order by inserting each subsequent array item into its correct position relative to the preceding ones.

The Insertion Sort Algorithm

When the last item has been placed, the entire array is in ascending order.

Figure 11.3.2 illustrates the action of step k of insertion on an array $a[1], a[2], a[3], \dots, a[n]$.



- Step k of Insertion Sort
- Figure 11.3.2

The Insertion Sort Algorithm

Algorithm 11.3.1 Insertion Sort

[The aim of this algorithm is to take an array $a[1], a[2], a[3], \dots, a[n]$, where $n \geq 1$, and reorder it. The output array is also denoted $a[1], a[2], a[3], \dots, a[n]$. It has the same values as the input array, but they are in ascending order. In the k th step, $a[1], a[2], a[3], \dots, a[k-1]$ is in ascending order, and $a[k]$ is inserted into the correct position with respect to it.]

Input: n [a positive integer], $a[1], a[2], a[3], \dots, a[n]$ [an array of data items capable of being ordered]

The Insertion Sort Algorithm

continued

Algorithm 11.3.1 Insertion Sort

Algorithm Body:

for $k := 2$ **to** n

[Compare $a[k]$ to previous items in the array $a[1], a[2], a[3], \dots, a[k-1]$, starting from the largest and moving downward. Whenever $a[k]$ is less than a preceding array item, the indexes of $a[k]$ and the preceding item are switched. As soon as $a[k]$ is greater than or equal to an array item, the value of $a[k]$ is left unchanged.]

$x := a[k]$

$j := k - 1$

while $(j \neq 0)$

if $x < a[j]$ **then**

$a[j + 1] := a[j]$

$a[j] := x$

$j := j - 1$

else $j := 0$

end if

end while

next k

Output: $a[1], a[2], a[3], \dots, a[n]$ [in ascending order]

Example 11.3.5 – A Trace Table for Insertion Sort

Construct a trace table showing the action of insertion sort on the array

$$a[1] = 6, a[2] = 3, a[3] = 5, a[4] = 7, a[5] = 2.$$

Example 11.3.5 – Solution

The first column shows the state of the variables before the first iteration of the for-next loop.

<i>n</i>	5											
<i>a</i> [1]	6	3										2
<i>a</i> [2]	3	6		5							2	3
<i>a</i> [3]	5			6						2	5	
<i>a</i> [4]	7								2	6		
<i>a</i> [5]	2								7			
<i>k</i>	2		3			4		5				
<i>x</i>	3		5			7		2				
<i>j</i>	1	0	2	1	0	3	0	4	3	2	1	0

Example 11.3.5 – Solution

When the for-next loop is first iterated, k is assigned the value 2; x the value of $a[2]$, which is 3; and j the value of $k - 1$, which is 1.

Because $j \neq 0$, the while loop is entered and the condition for the if-then-else statement is tested.

Because $a[1] > x$, then $a[2]$ is assigned the value of $a[1]$, which is 6, j is assigned the value of $j - 1$, which is 0, and $a[1]$ is assigned the value of x , which is 3.

Example 11.3.5 – Solution

The condition governing the while loop is tested again, but since $j = 0$, it is not satisfied, and so the while loop is not entered.

Then the value of k is incremented by 1 (so that it equals 3), and the for-next loop is entered a second time. This process continues until the value of k has been incremented to 6.

Because 6 is greater than the top value in the for-next loop, execution of the algorithm ceases, and the array items are in ascending order.

Example 11.3.6 – Finding a Worst-Case Order for Insertion Sort

- a. When insertion sort is applied to the array $a[1], a[2], a[3], \dots, a[n]$, what is the maximum number of comparisons if those that control the while and for-next loops are included in the count?

- b. Use the theorem on polynomial orders to find a worst-case order for insertion sort.

Example 11.3.7 – Finding an Average-Case Order for Insertion Sort

- a. What is the average number of comparisons that are performed when insertion sort is applied to the array $a[1], a[2], a[3], \dots, a[n]$?

- b. Use the theorem on polynomial orders to find an average-case order for insertion sort.

Outline

1. Big-O, Big-Omega, and Big-Theta Notations
2. Application: Analysis of Algorithm Efficiency: I
3. Application: Analysis of Algorithm Efficiency : II

Application: Analysis of Algorithm Efficiency II

Have you ever played the “guess my number” game? A person thinks of a number between two other numbers—say, 1 and 10 or 1 and 100 for example—and you try to figure out what it is, using the least possible number of guesses. Each time you guess a number, the person tells you whether you are correct, too low, or too high.

If you have played this game, you have probably already hit upon the most efficient strategy: Begin by guessing a number as close to the middle of the two given numbers as possible.

Application: Analysis of Algorithm Efficiency II

If your guess is too high, then the number is between the lower of the two given numbers and the one you first chose. If your guess is too low, then the number is between the number you first chose and the higher of the two given numbers.

In either case, you take as your next guess a number as close as possible to the middle of the new range in which you now know the number lies. You repeat this process as many times as necessary until you have found the person's number.

Application: Analysis of Algorithm Efficiency II

The technique described previously is an example of a general strategy called **divide and conquer**, which works as follows: To solve a problem, reduce it to a fixed number of smaller problems of the same kind, which can themselves be reduced to the same fixed number of smaller problems of the same kind, and so forth until easily resolved problems are obtained.

In this case, the problem of finding a particular number in a given range of numbers is reduced at each stage to finding a particular number in a range of numbers approximately half as long.

Application: Analysis of Algorithm Efficiency II

It turns out that algorithms using a **divide-and-conquer** strategy are generally quite efficient and nearly always have orders involving logarithmic functions.

In this section we define the binary search algorithm, which is the formalization of the “guess my number” game described previously, and we compare the efficiency of binary search to the sequential search. Then we develop a divide-and-conquer algorithm for sorting, merge sort, and compare its efficiency with that of insertion sort and selection sort.

Application: Analysis of Algorithm Efficiency II

Almost a hundred years before the first computer was built and working only from a description of how such a machine might function, Ada Augusta, Countess of Lovelace, foresaw both the general nature of the problems it could address and the importance of designing an efficient arrangement of computations to solve them.

Binary Search

Whereas a sequential search can be performed on an array whose elements are in any order, a binary search can be performed only on an array whose elements are arranged in ascending (or descending) order. Given an array $a[1], a[2], \dots, a[n]$ of distinct elements arranged in ascending order, consider the problem of trying to find a particular element x in the array.

To use binary search, first compare x to the “middle element” of the array. If the two are equal, the search is successful.

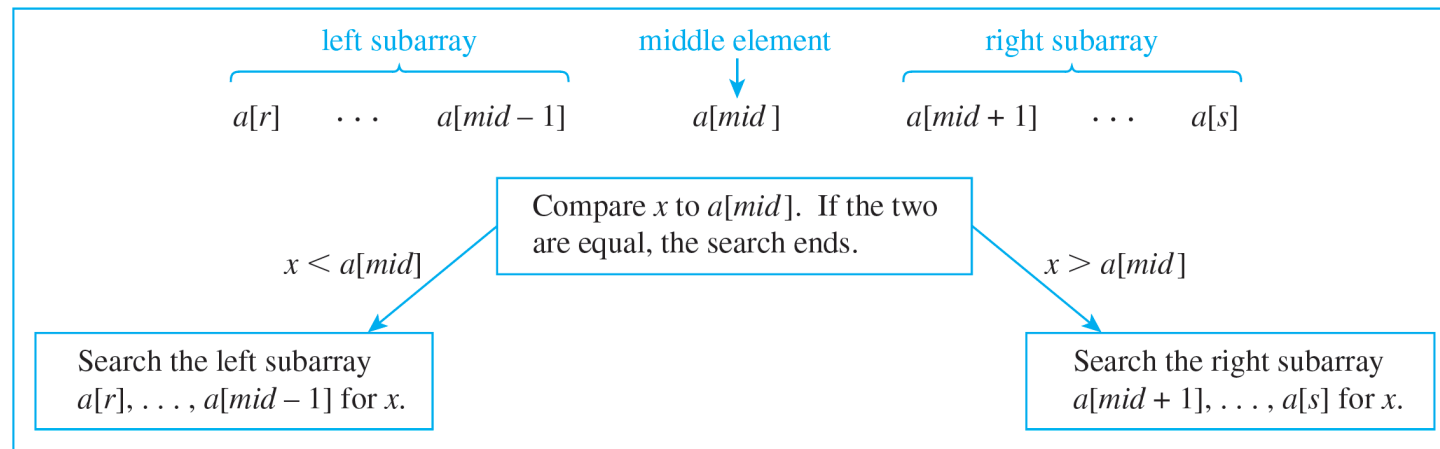
Binary Search

If the two are not equal, then because the array elements are in ascending order, comparing the values of x and the middle array element narrows the search either to the lower subarray (consisting of all the array elements below the middle element) or to the upper subarray (consisting of all array elements above the middle element).

The search continues by repeating this basic process over and over on successively smaller subarrays. It terminates either when a match occurs or when the subarray to which the search has been narrowed contains no elements.

Binary Search

The efficiency of the algorithm is a result of the fact that at each step, the length of the subarray to be searched is roughly half the length of the array of the previous step. This process is illustrated in Figure 11.5.1.



•One Iteration of the Binary Search Process

•Figure 11.5.1

Binary Search

Algorithm 11.5.1 Binary Search

[The aim of this algorithm is to search for an element x in an ascending array of elements $a[1], a[2], \dots, a[n]$. If x is found, the variable $index$ is set equal to the index of the array element where x is located. If x is not found, $index$ is not changed from its initial value, which is 0. The variables bot and top denote the bottom and top indexes of the array currently being examined.]

Input: n [a positive integer], $a[1], a[2], \dots, a[n]$ [an array of data items given in ascending order], x [a data item of the same data type as the elements of the array]

Binary Search

Algorithm 11.5.1 Binary Search

Algorithm Body:

$index := 0, bot := 1, top := n$

[Compute the middle index of the array, mid . Compare x to $a[mid]$. If the two are equal, the search is successful. If not, repeat the process either for the lower or for the upper subarray, either giving top the new value $mid - 1$ or giving bot the new value $mid + 1$. Each iteration of the loop either decreases the value of top or increases the value of bot . Thus, if the looping is not stopped by success in the search process, the value of top eventually becomes less than the value of bot , which stops the looping process and shows that x is not an element of the array.]

while ($top \geq bot$ and $index = 0$)

$$mid := \left\lfloor \frac{bot + top}{2} \right\rfloor$$

if $a[mid] = x$ **then** $index := mid$

if $a[mid] > x$

then $top := mid - 1$

else $bot := mid + 1$

end while

*[If $index$ has the value 0 at this point, then the **while** loop was not entered because $top < bot$, and so x is not in the array. Otherwise, $index$ gives the index of the array where x is located.]*

Output: $index$ [a nonnegative integer]

Example 11.5.1 – Tracing the Binary Search Algorithm

Trace the action of Algorithm 11.5.1 on the variables index, bot, top, mid, and the values of x given in (a) and (b) below for the input array

a[1] = Ann, a[2] = Dawn, a[3] = Erik, a[4] = Gail, a[5] = Juan,
a[6] = Matt, a[7] = Max, a[8] = Rita, a[9] = Tsuji, a[10] = Yuen,

where alphabetical ordering is used to compare elements of the array.

a. x = Erik

b. x = Sara

Example 11.5.1 – Solution

a.

<i>index</i>	0			3
<i>bot</i>	1		3	
<i>top</i>	10	4		
<i>mid</i>		5	2	3
<i>x</i>	Erik			

b.

<i>index</i>	0			
<i>bot</i>	1	6	9	
<i>top</i>	10			8
<i>mid</i>		5	8	9
<i>x</i>	Sara			

The Efficiency of the Binary Search Algorithm

Here briefly is the idea for how to derive the efficiency of the binary search algorithm. At each stage of the binary search process, the length of the new subarray to be searched is approximately half that of the previous one, and in the worst case, every subarray down to a subarray with a single element must be searched.

Consequently, in the worst case, the maximum number of iterations of the algorithm's while loop is 1 more than the number of times the original input array can be cut approximately in half.

The Efficiency of the Binary Search Algorithm

Now if the length n of this array is a power of 2 ($n = 2^k$ for some integer k), then n can be halved exactly $k = \log_2 n = \lfloor \log_2 n \rfloor$ times before an array of length 1 is reached. If n is not a power of 2, then $n = 2^k + m$ for some integer k (where $m < 2^k$), and thus n can be split approximately in half k times also, and so in this case, $k = \lfloor \log_2 n \rfloor$ also.

Thus in the worst case, the maximum number of iterations of the **while** loop in the binary search algorithm (which is proportional to the number of comparisons required to execute it) is $\lfloor \log_2 n \rfloor + 1$.

The Efficiency of the Binary Search Algorithm

The derivation is concluded by noting that $\lfloor \log_2 n \rfloor + 1$ is $O(\log_2 n)$.

The details of the derivation are developed in Examples 11.5.2-11.5.6.

Throughout the derivation, for each integer $n \geq 1$, let

w_n = the number of iterations of the **while** loop
in a *worst-case* execution of the binary search
algorithm for an input array of length n .

The first issue to consider is this. If the length of the input array for one iteration of the **while** loop is known, what is the greatest possible length of the array input to the next iteration?

Example 11.5.2 – The Length of the Input Array to the Next Iteration of the Loop

Prove that if an array of length k is input to the while loop of the binary search algorithm, then after one unsuccessful iteration of the loop, the input to the next iteration is an array of length at most $\lfloor k/2 \rfloor$.

Example 11.5.3 – A Recurrence Relation for $w_1, w_2, w_3, \dots$

Prove that the sequence $w_1, w_2, w_3, \dots$ satisfies the recurrence relation and initial condition

$$w_1 = 1, w_k = 1 + w_{\lfloor k/2 \rfloor} \text{ for every integer } k > 1.$$

Example 11.5.4 – An Explicit Formula for $w_1, w_2, w_3, \dots$

Apply iteration to the recurrence relation found in Example 11.5.3 to conjecture an explicit formula for $w_1, w_2, w_3, \dots$

Example 11.5.6 – The Binary Search Algorithm Is Logarithmic

By Example 11.5.5,

$$w_n = \lfloor \log_2 n \rfloor + 1 \quad \text{for every integer } n \geq 1.$$

Use this result to show that in the worst case, the binary search algorithm is $\Theta(\log_2 n)$.

Tractable and Intractable Problems

At an opposite extreme from an algorithm such as binary search, which has logarithmic order, is an algorithm with exponential order.

For example, consider an algorithm to direct the movement of each of the 64 disks in the Tower of Hanoi puzzle as they are transferred one by one from one pole to another. We know that such a transfer requires $2^{64} - 1$ steps.

Tractable and Intractable Problems

If a computer took a nanosecond to calculate each transfer step, the total time to calculate all the steps would be

$$(2^{64} - 1) \cdot \left(\frac{1}{10^9}\right) \cdot \left(\frac{1}{60}\right) \cdot \left(\frac{1}{60}\right) \cdot \left(\frac{1}{24}\right) \cdot \left(\frac{1}{365.25}\right) \approx 584.5 \text{ years.}$$

↑
number of
moves

moves
per
second

seconds
per
minute

minutes
per
hour

hours
per
day

days
per
year

Tractable and Intractable Problems

Problems whose solutions can be found with algorithms whose worst-case order with respect to time is a polynomial are said to belong to **class P**. They are called **polynomial-time algorithms** and are said to be **tractable**.

Problems that cannot be solved in polynomial time are called **intractable**. For certain problems, it is possible to check the correctness of a proposed solution with a polynomial-time algorithm, but it may not be possible to find a solution in polynomial time.

Tractable and Intractable Problems

Such problems are said to belong to **class NP**. The biggest open question in theoretical computer science is whether every problem in class NP belongs to class P.

This is known as the **P vs. NP** problem. The Clay Institute, in Cambridge, Massachusetts, has offered a prize of \$1,000,000 to anyone who can either prove or disprove that $P = NP$.

Tractable and Intractable Problems

In recent years, computer scientists have identified a fairly large set of problems, called **NP-complete**, that all belong to class NP but are widely believed not to belong to class P.

What is known for sure is that if any one of these problems is solvable in polynomial time, then so are all the others. One of the NP-complete problems, commonly known as the *traveling salesman problem*.